

图应用 优先级搜索

通用算法

❖ 各种遍历算法的区别，仅在于选取顶点进行访问的**次序**

广度/深度：优先访问与**更早/更晚**被发现的顶点相邻接者；...

❖ 不同的遍历算法，取决于顶点的**选取策略**

❖ 不同的顶点选取策略，取决于存放和提供顶点的**数据结构**——Bag

❖ 此类结构，为每个顶点 v 维护一个**优先级数**—— $\text{priority}(v)$

- 每个顶点都有**初始**优先级数；并可能随算法的推进而**调整**

❖ 通常的习惯是，优先级数越**大/小**，优先级越**低/高**

- 特别地， $\text{priority}(v) == \text{INT_MAX}$ ，意味着 v 的优先级最低

统一框架 (1/2)

```
template <typename Tv, typename Te>

template <typename PU> //优先级更新器 (函数对象)

void Graph<Tv, Te>::pfs( Rank s, PU prioUpdater ) { //PU的策略, 因算法而异

    priority(s) = 0; status(s) = VISITED; parent(s) = -1; //起点s加至PFS树中

    while (1) { //将下一顶点和边加至PFS树中

        /* ... 依次引入n-1个顶点 (和n-1条边) ... */

    } //while

} //如何推广至非连通图?
```

统一框架 (2/2)

```
while (1) { //依次引入n - 1个顶点 (和n - 1条边)

    for ( Rank w = firstNbr(s); -1 < w; w = nextNbr(s, w) ) //对s各邻居w

        prioUpdater( this, s, w ); //更新顶点w的优先级及其父顶点

    for ( int shortest = INT_MAX, w = 0; w < n; w++ )

        if ( UNDISCOVERED == status(w) ) //从尚未加入遍历树的顶点中

            if ( shortest > priority(w) ) //选出下一个

                { shortest = priority(w); s = w; } //优先级最高的顶点s

    if ( VISITED == status(s) ) break; //直至所有顶点均已加入

    status(s) = VISITED; type( parent(s), s ) = TREE; //将s加入遍历树

} //while
```

复杂度

- ❖ 执行时间主要消耗于内、外两重循环；其中两个内循环前、后并列
- ❖ 前一内循环的累计执行时间：若采用邻接矩阵，为 $O(n^2)$ ；若采用邻接表，为 $O(n+e)$
后一循环中，优先级更新的次数呈算术级数变化 $\{ n, n-1, \dots, 2, 1 \}$ ，累计为 $O(n^2)$
两项合计，为 $O(n^2)$
- ❖ 后面将会看到：若采用优先级队列，以上两项将分别是 $O(e \log n)$ 和 $O(n \log n)$ //保持兴趣
两项合计，为 $O((e+n) \log n)$
- ❖ 这是很大的改进——尽管对于稠密图而言，反而是倒退 //已有接近于 $O(e + n \log n)$ 的算法
- ❖ 基于这个统一框架，如何解决具体的应用问题...

图应用

Dijkstra算法： 最短路径

问题 + 应用

❖ 给定：连通有向图 G 及其中的顶点 u 和 v

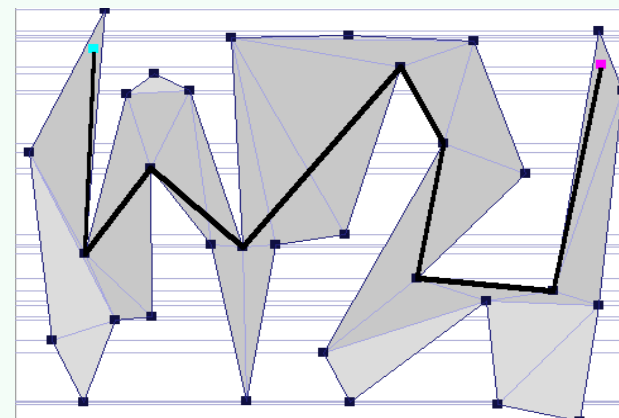
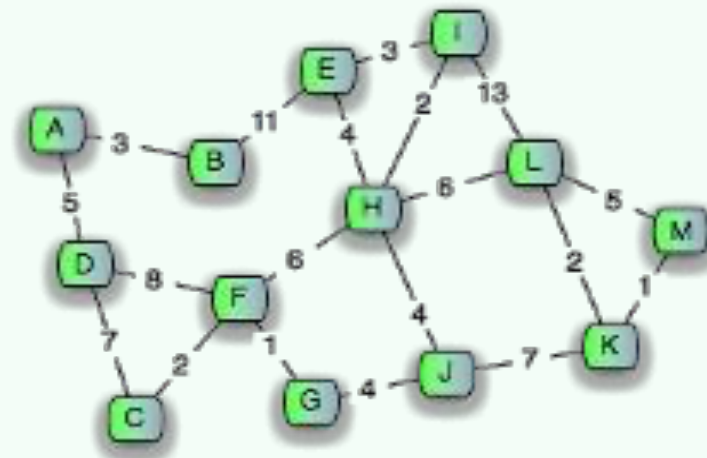
找到：从 u 到 v 的最短路径及其长度

❖ 旅游者：最经济的出行路线

路由器：最快地将数据包传送到目标位置

路径规划：多边形区域内的自主机器人

.....



问题分类

- ❖ 按照图的类型：无（等）权图（BFS）；带权有向图 //权值非负？
- ❖ 【E. Dijkstra, 1959】SSSP: Single-Source Shortest Path

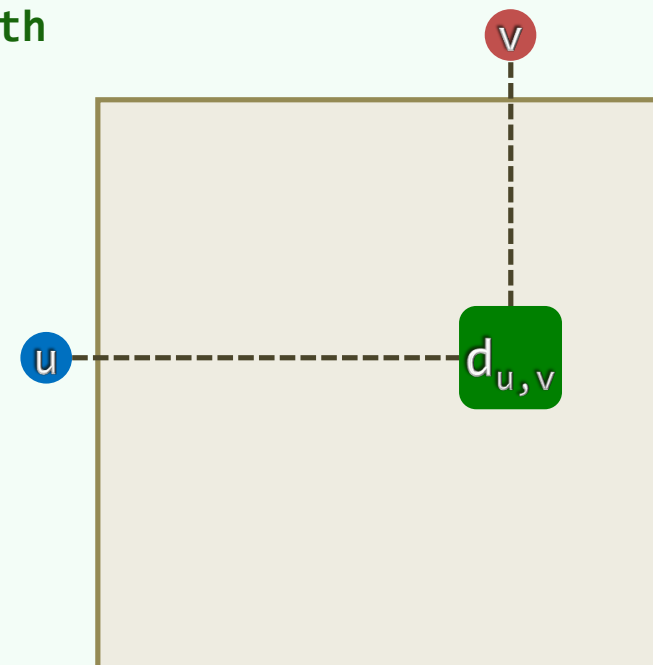
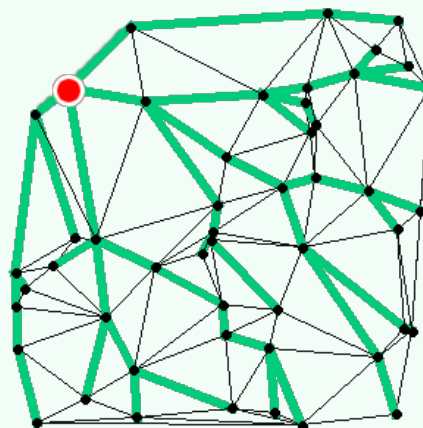
给定顶点 s ，计算 s 到

其余各个顶点的最短路径及长度

- ❖ 【Floyd-Warshall, 1962】

APSP: All-Pairs Shortest Path

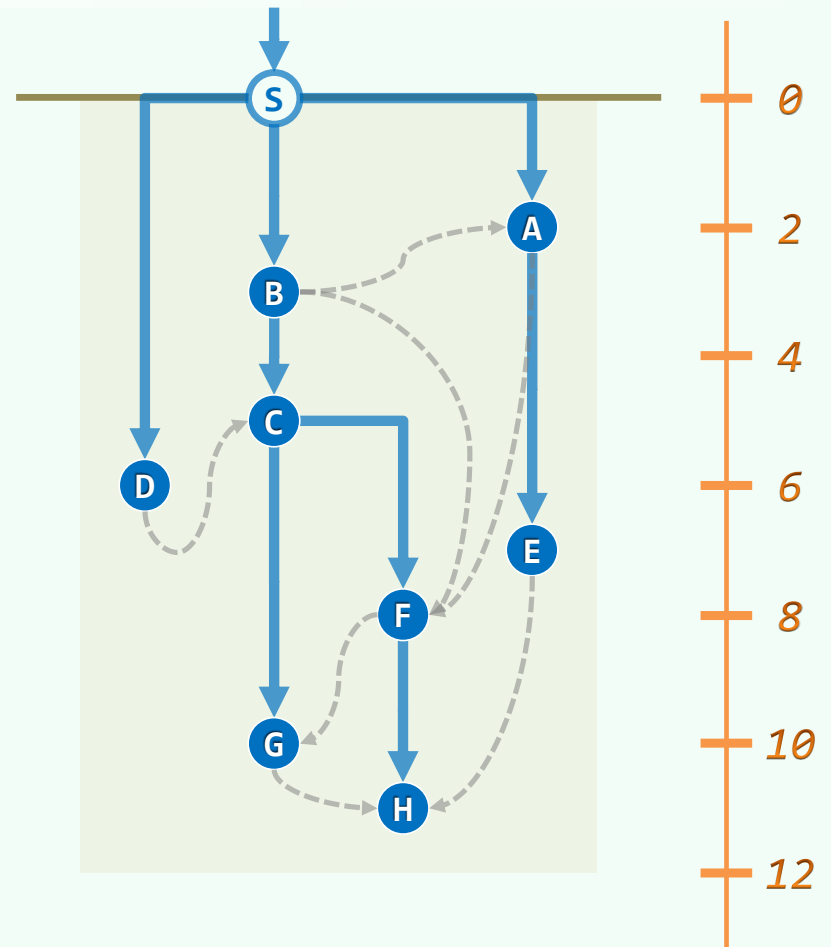
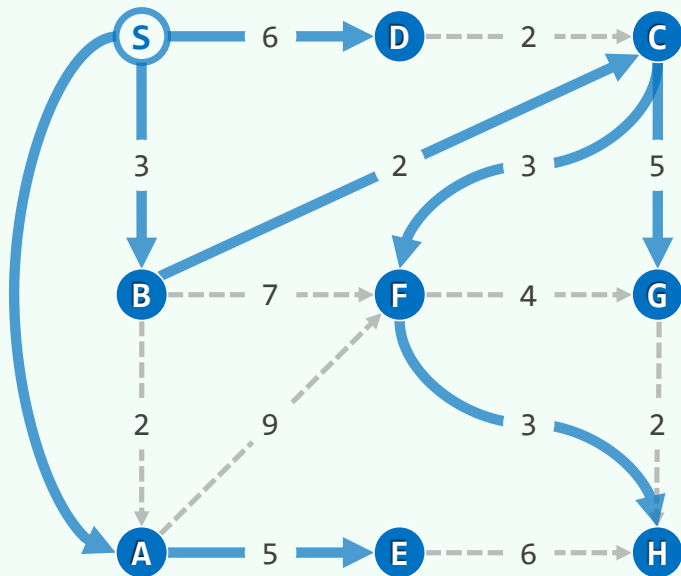
找出每对顶点 u 和 v 之间的最短路径及长度



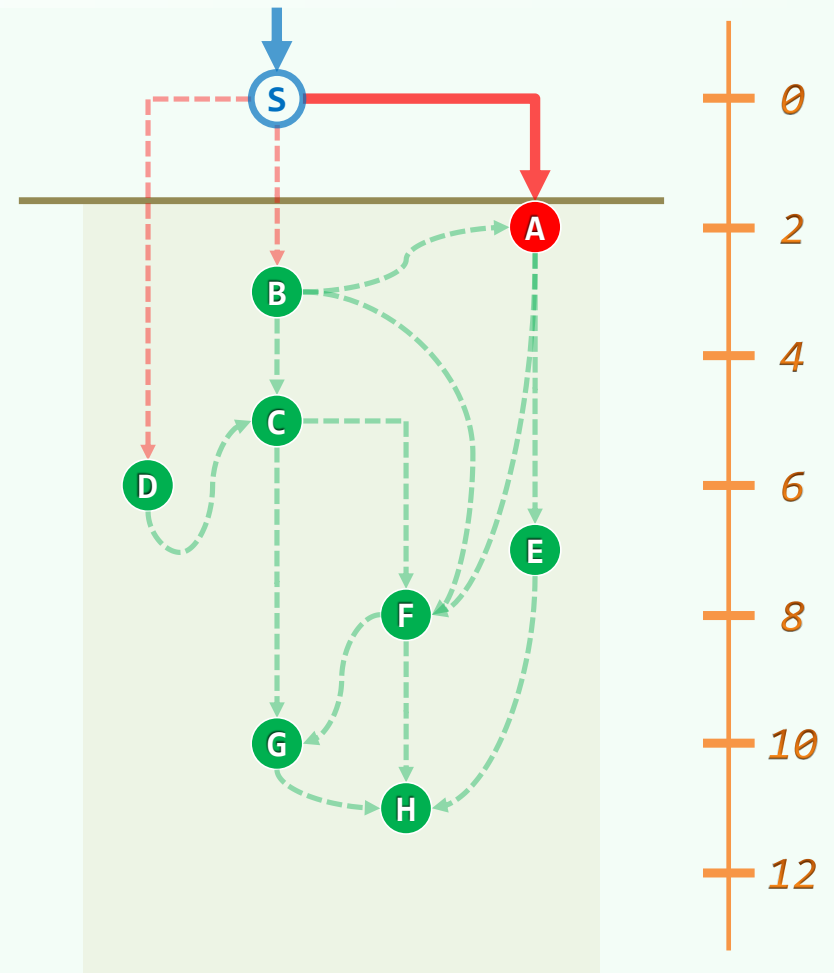
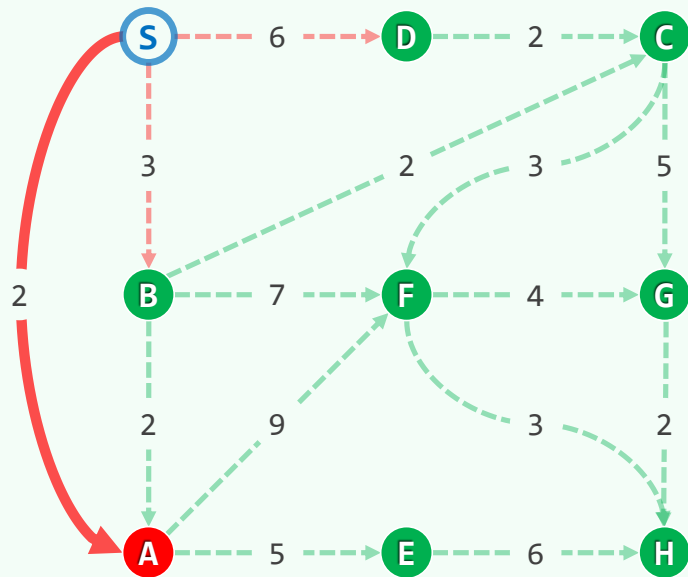
图应用

Dijkstra算法： 算法

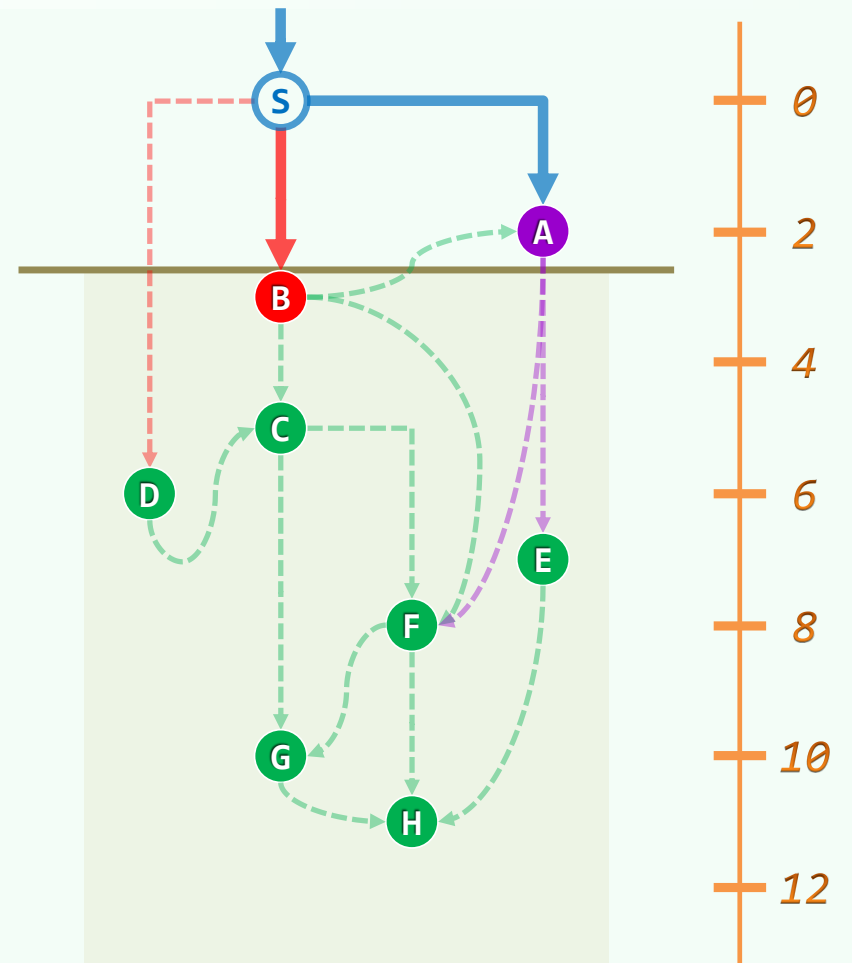
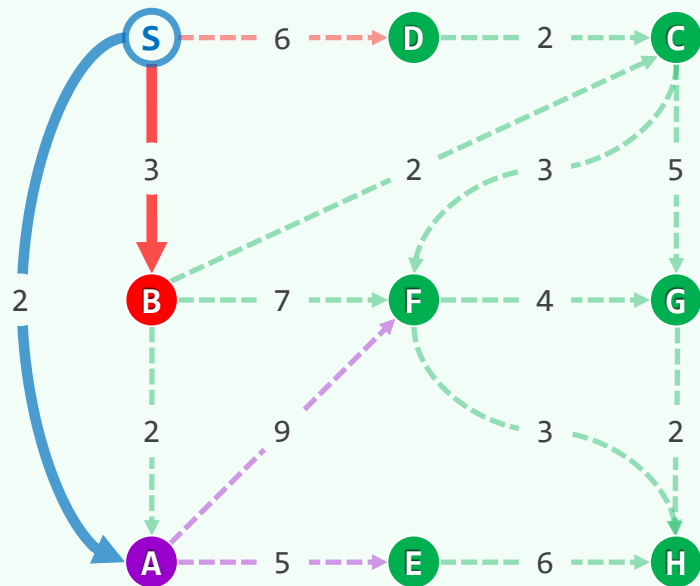
减而治之 = 遍历 = PFS



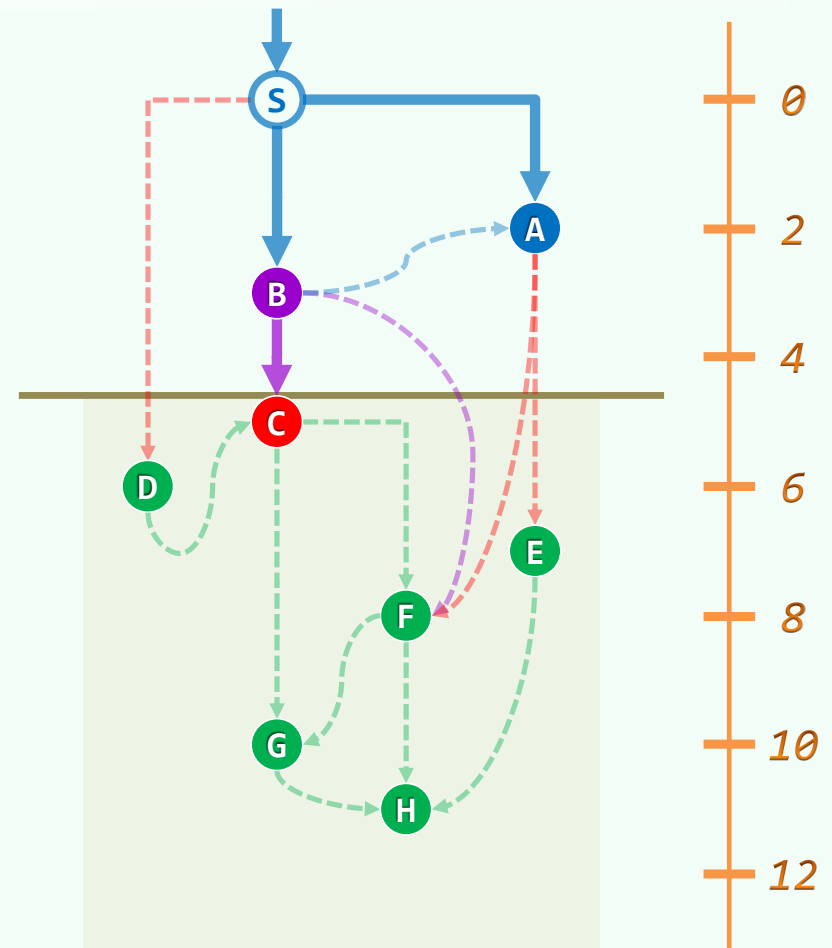
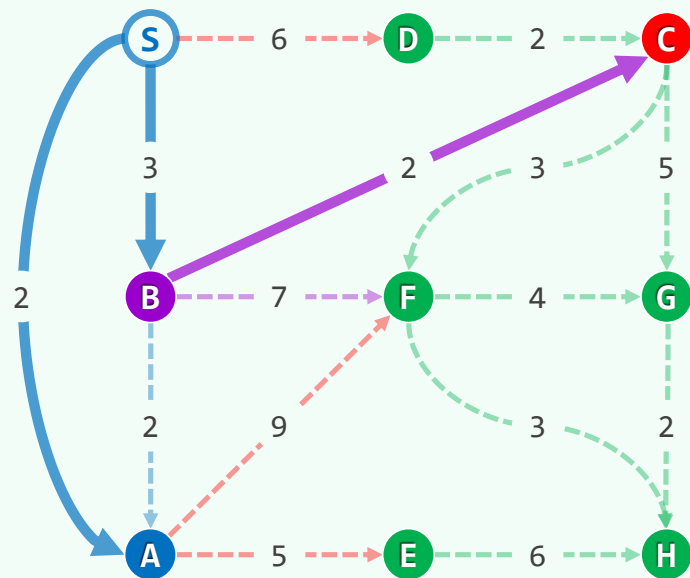
S起步，确定A



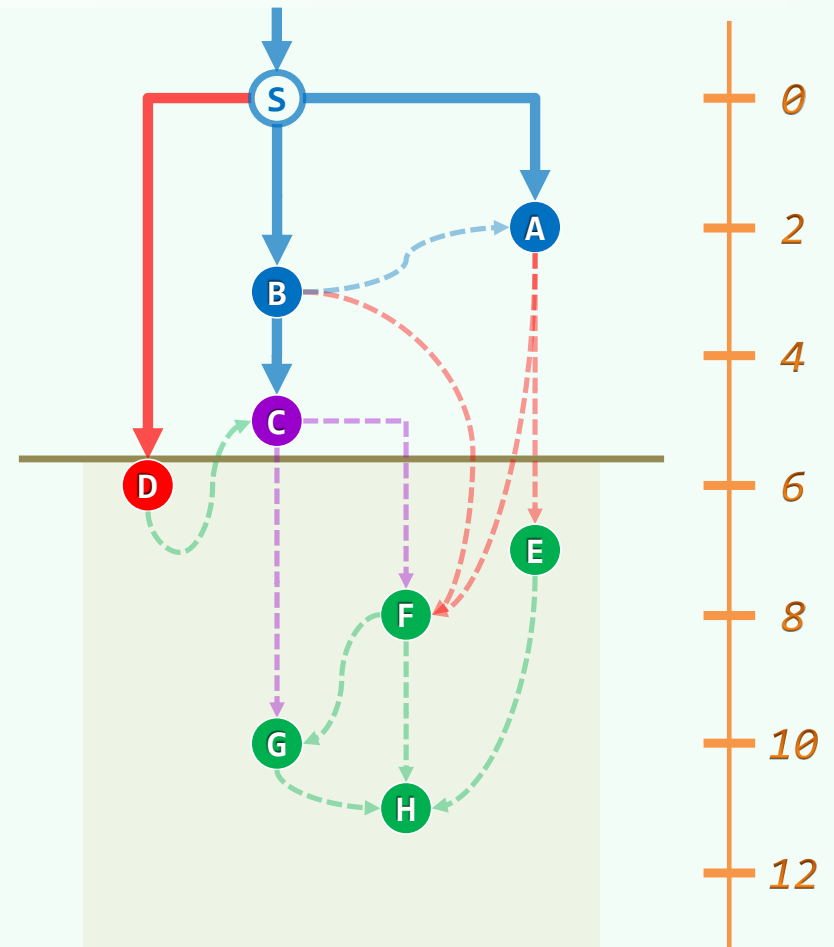
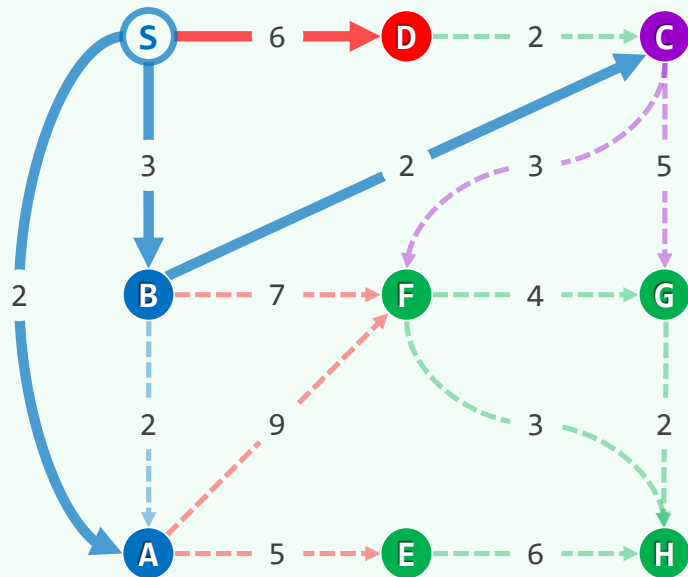
A入选，确定B



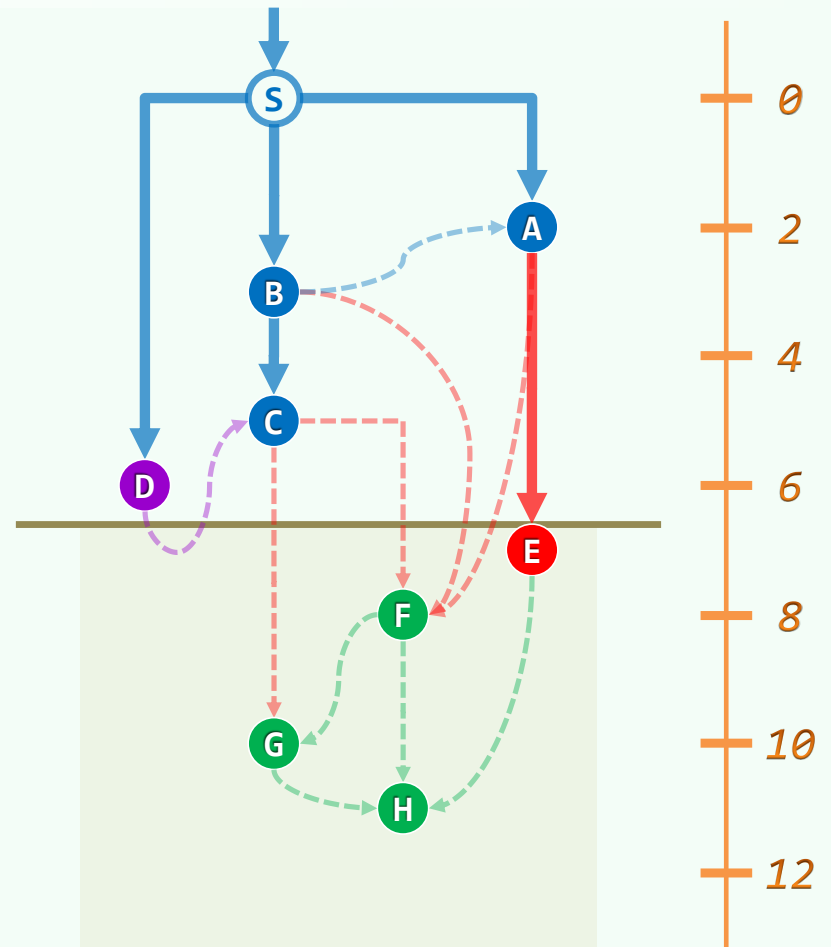
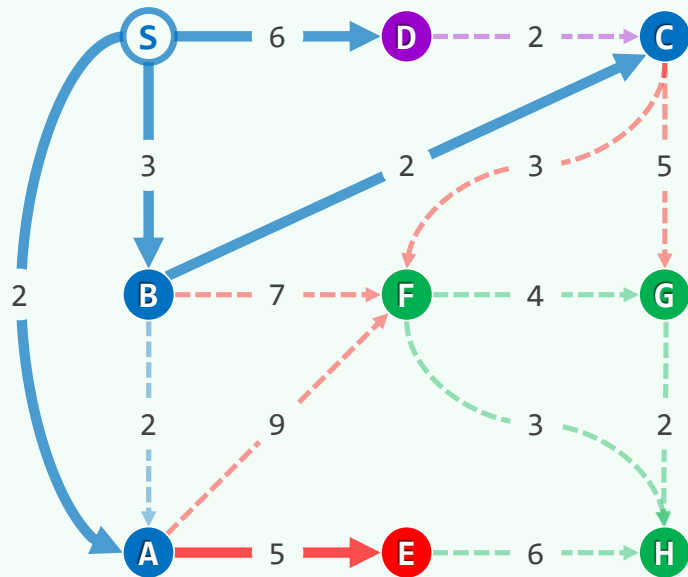
B入选，确定C



C入选，确定D



D入选，确定E



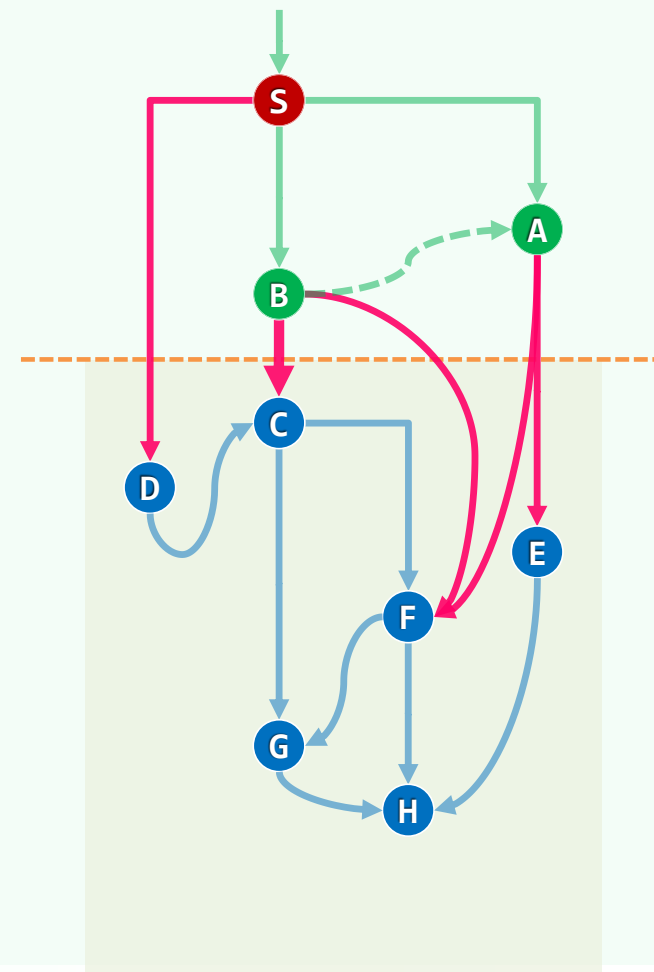
图应用

Dijkstra算法：实现

PFS

- ❖ $\forall v \notin V_k$, let $\text{priority}(v) = \|s, v\| \leq \infty$
- ❖ 于是套用PFS框架，为将 T_k 扩充至 T_{k+1} ，只需
 - 选出优先级最高的跨边 e_k 及其对应顶点 u_k ，并将其加入 T_k
 - 随后，更新 $V \setminus V_{k+1}$ 中所有顶点的优先级（数）
- ❖ 注意：优先级数随后可能改变（降低）的顶点，必与 u_k 邻接
- ❖ 因此，只需枚举 u_k 的每一邻接顶点 v ，并取

$$\text{priority}(v) = \min(\text{priority}(v), \text{priority}(u_k) + \|u_k, v\|)$$
- ❖ 以上完全符合PFS的框架，唯一要做的工作无非是
按照prioUpdater()规范，编写一个优先级（数）更新器...



PrioUpdater()

```
g->pfs( 0, DijkPU<char, Rank>() ); //从顶点0出发, 启动Dijkstra算法

template <typename Tv, typename Te> struct DijkPU { //Dijkstra算法的顶点优先级更新器

    virtual void operator()( Graph<Tv, Te>* g, Rank s, Rank w ) { // 对s的每个

        if ( UNDISCOVERED != g->status(w) ) return; //尚未被发现的邻居w, 按

        if ( g->priority(w) > g->priority(s) + g->weight(s, w) ) { //Dijkstra

            g->priority(w) = g->priority(s) + g->weight(s, w); //策略

            g->parent(w)   = s; //做松弛

        }

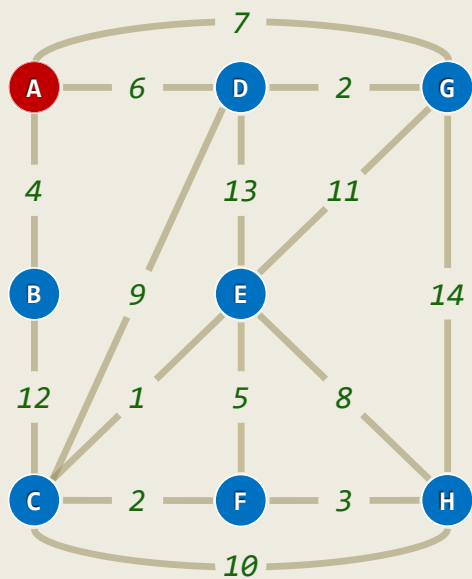
    }

};
```

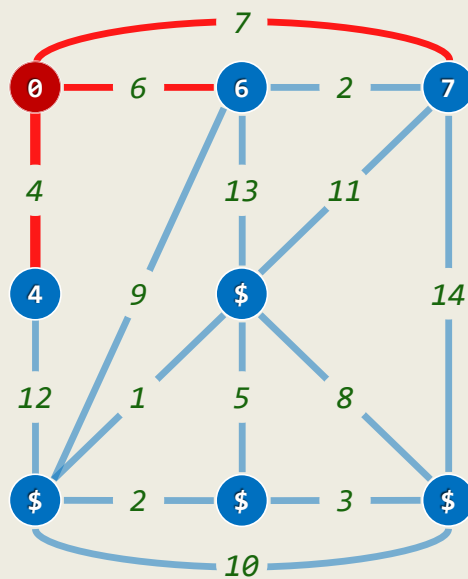
图应用

Dijkstra算法：实例

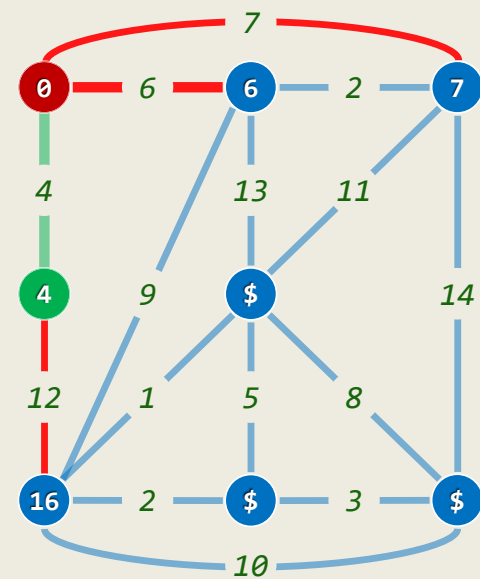
1/4



(a)

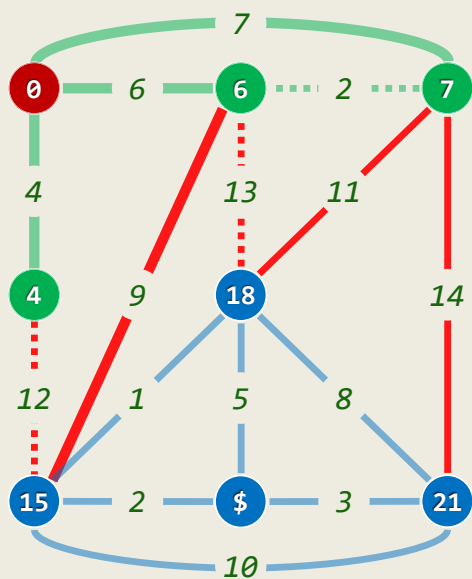


(b)

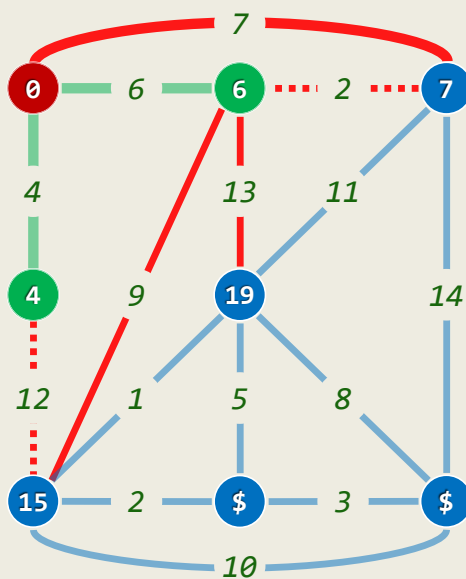


(c)

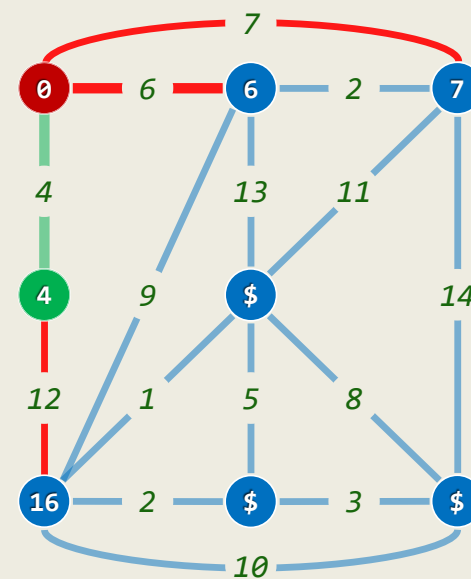
2/4



(e)

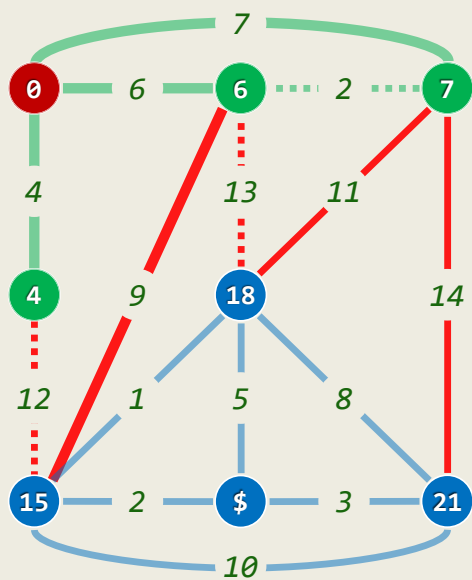


(d)

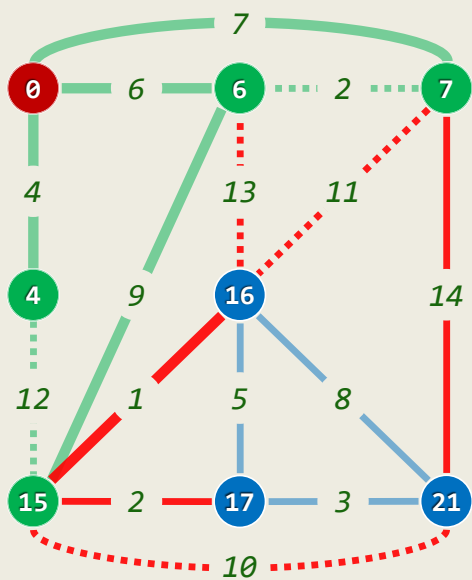


(c)

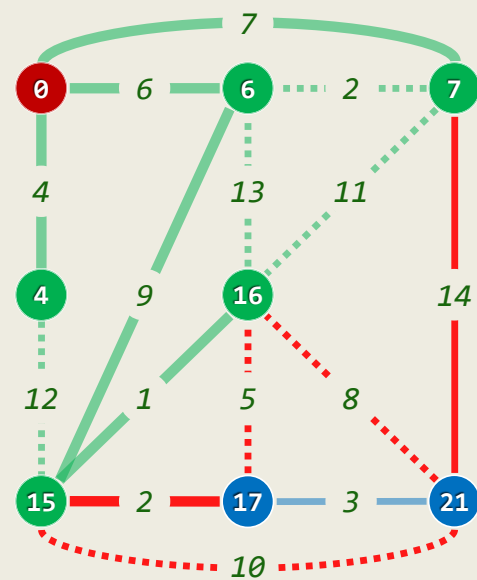
3/4



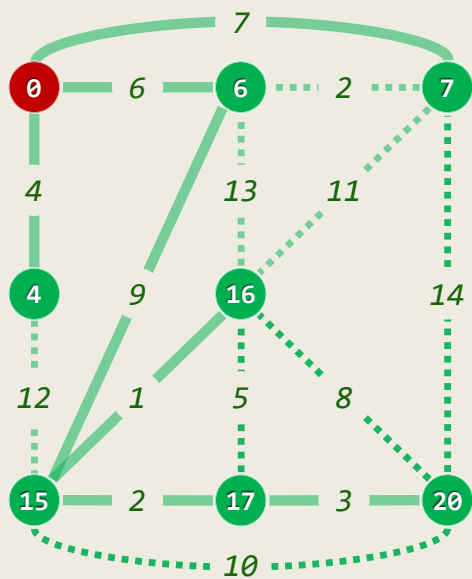
(e)



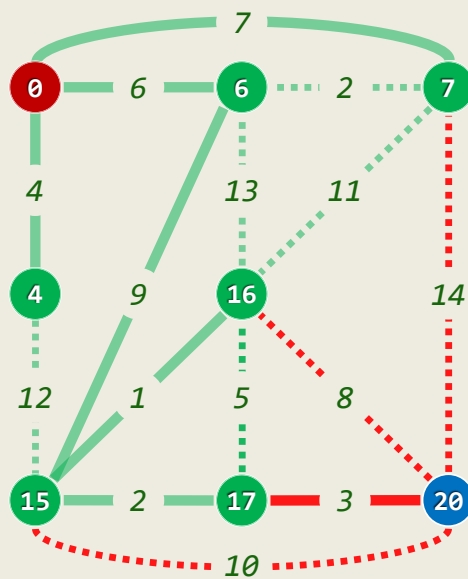
(f)



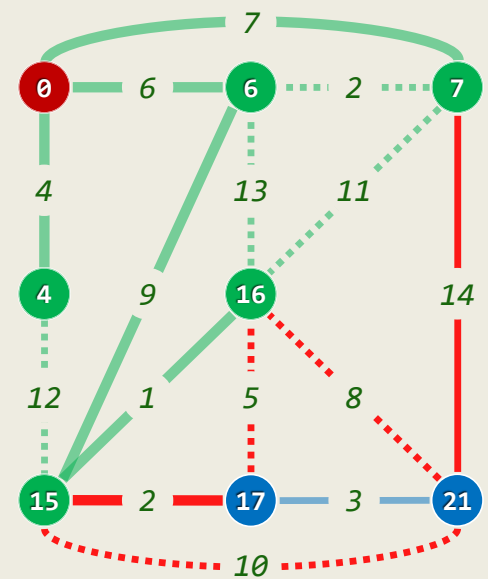
(g)



(i)



(h)



(g)

图应用

Floyd-Warshall算法

从Dijkstra到Floyd-Warshall

❖ 给定带权网络 G ，计算其中**所有点对**之间的最短距离

❖ 应用：确定 G 的中心点 (center)

- $\text{radius}(G, s) = s$ 的半径 = 所有顶点到 s 的最大距离

- 中心点 = 半径最小的顶点 s

❖ 直觉：依次将各顶点作为源点，调用Dijkstra算法

时间 = $n \times O(n^2) = O(n^3)$ —— 可否更快？

❖ 思路：图矩阵 ~ 最短路径矩阵

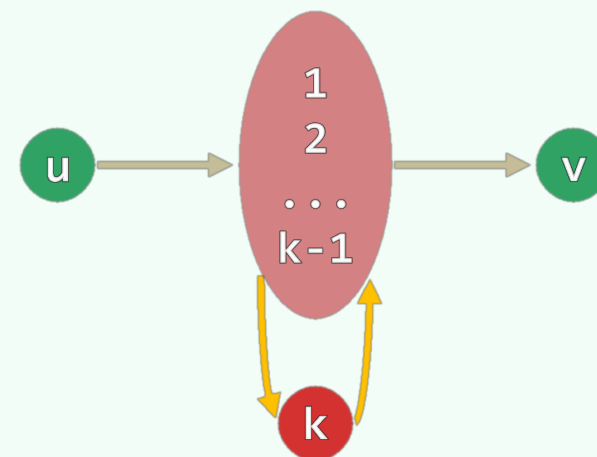
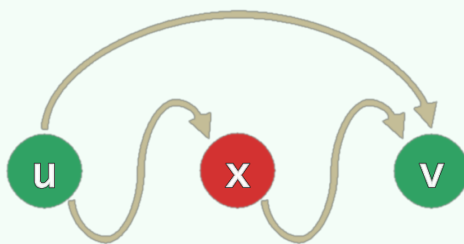
❖ 效率： $O(n^3)$ ，与执行 n 次Dijkstra相同 —— 既如此，为何还要用F.W.？

❖ 优点：形式简单、算法紧凑、便于实现；允许**负权边**（尽管仍不能有**负权环路**）

问题 + 特点

❖ u 和 v 之间的最短路径可能是

- 不存在通路，或者
- 直接连接，或者
- 最短路径(u, x) + 最短路径(x, v)



❖ 将所有顶点随意编号: $1, 2, \dots, n$

定义: $d^k(u, v)$ = 中途只经过前 k 个顶点中转, 从 u 通往 v 的最短路径长度

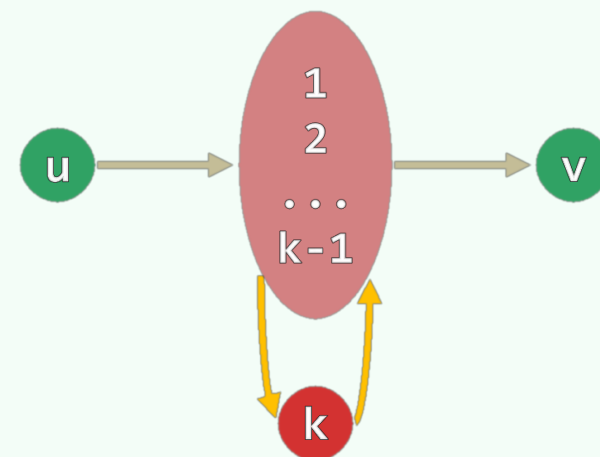
- $d^k(u, v) = w(u, v)$ (if $k = 0$)
- $d^k(u, v) = \min\{ d^{k-1}(u, v), d^{k-1}(u, k) + d^{k-1}(k, v) \}$ (if $k \geq 1$)

蛮力递归

```
❖ weight dist( node * u, node * v, int k )  
    if ( k < 1 ) return w( u, v );  
    u2v = dist( u, v, k-1 ); //经前k-1个点中转  
    for each node x  $\notin \{ u, v \}$  //x作为第k个可中转点  
        u2x2v = dist( u, x, k-1 )  
            + dist( x, v, k-1 ); //递归  
        u2v = min( u2v, u2x2v ); //择优  
    return u2v;
```

❖ 存在大量重复的递归调用，如何避免？

❖ 动态规划之记忆化：维护一张表，记录需要反复计算的数值



动态规划

```
❖ for k in range(0, n)
    for u in range(0, n)
        for v in range(0, n)
            A[u][v] =
                min( A[u][v], A[u][k] + A[k][v] )
```

